

# Nube de cargas de Fibonacci aplicada a la molécula de agua: integración numérica, derivación numérica y splines cúbicos, con validación frente a Excel–ScienSolar

Cristian Alberto Vargas Vera\*  
Universidad De Pamplona  
Departamento de Ciencias Básicas  
Física Computacional I  
(Dated: June 19, 2026)

Se reimplementa en Python un modelo de nube de cargas puntuales (*finite-point charge cloud*), desarrollado originalmente en Excel–ScienSolar, en el cual cada partícula cargada se representa mediante una distribución cuasi-uniforme de Fibonacci de  $N_c$  subcargas sobre una esfera, conservando el formalismo de superposición de Coulomb. Como caso de estudio se construye la molécula de agua a partir de la geometría experimental y las cargas parciales TIP3P. Sobre este modelo se aplican tres métodos numéricos: cuadratura de Gauss–Legendre (para verificar la conservación de la carga total de cada nube), diferencias finitas centradas (para obtener el campo eléctrico a partir del potencial) y splines cúbicos (para reconstruir un perfil radial continuo a partir de un muestreo disperso). Los resultados se validan frente a los reportados en Excel–ScienSolar, con acuerdo del orden de  $10^{-4}$  %, y se compara el tiempo de ejecución de ambas plataformas. Las ecuaciones gobernantes y la justificación detallada de cada método numérico se presentan en el Apéndice.

**Keywords:** Discretización de Coulomb, nube de Fibonacci, molécula de agua, diferencias finitas, Gauss–Legendre, splines cúbicos, ScienSolar.

## I. INTRODUCCIÓN

El cálculo del potencial  $V(\mathbf{r})$  y el campo  $\mathbf{E}(\mathbf{r})$  generados por una distribución de carga  $\rho(\mathbf{r})$  requiere, en general, resolver una integral de Coulomb que solo admite solución analítica para geometrías simples. Una alternativa, propuesta en el modelo de Excel–ScienSolar [1, 2], consiste en *discretizar* cada partícula cargada en  $N_c$  subcargas puntuales distribuidas geométricamente, reemplazando la integral por una suma de Coulomb. Este trabajo reimplementa dicho modelo **desde cero en Python**, lo aplica a un sistema físicamente relevante –la molécula de agua– e incorpora explícitamente los tres métodos numéricos del curso: integración numérica (Gauss–Legendre), derivación numérica (diferencias finitas) e interpolación con splines cúbicos. La molécula de agua se eligió por tener tres centros de carga no colineales, lo que produce una anisotropía apreciable en campo cercano (dipolo) que converge al límite de carga puntual en campo lejano, ilustrando ambos regímenes discutidos en [1].

## II. MODELO Y MÉTODOS (RESUMEN)

Cada átomo de carga total  $Q_j$  se reemplaza por  $N_c$  subcargas  $q_{j,k} = Q_j/N_c$  distribuidas sobre una esfera de radio  $r_{\text{cloud}}$  mediante muestreo de **Fibonacci** (Ec. A2 del Apéndice). El potencial y el campo totales se calculan por superposición de Coulomb (Ec. A1). De las cinco geometrías propuestas en [1], se usa exclusivamente Fi-

bonacci por ser la que converge más rápido al límite de carga puntual.

La molécula de agua se construye con la geometría experimental ( $r_{OH} = 0.9572$  Å,  $\angle HOH = 104.5^\circ$ ) y cargas parciales TIP3P ( $Q_O = -0.834e$ ,  $Q_H = +0.417e$  cada hidrógeno), discretizando cada átomo con  $N_c = 400$  y  $r_{\text{cloud}} = 0.30$  Å. La constante de Coulomb se fija a  $K_{\text{eff}} = 14.3800$  VÅ/ $e$ , el valor exacto reportado en la Tabla 2 de [1], lo que permite usar esa tabla como prueba unitaria del código.

Se incorporan tres métodos numéricos, cuya formulación completa y justificación detallada se presentan en el Apéndice A: (i) **integración numérica** mediante cuadratura de Gauss–Legendre, para verificar que la nube discreta conserva la carga total del átomo; (ii) **derivación numérica** mediante diferencias finitas centradas, para obtener  $\mathbf{E}$  a partir de  $V$ ; y (iii) **splines cúbicos**, para reconstruir un perfil continuo  $V(r)$  a partir de un muestreo disperso de solo 8 puntos.

## III. RESULTADOS Y DISCUSIÓN

La implementación en Python (`agua_cloud_final.py`) sigue cuatro etapas –generación de geometría, evaluación de Coulomb, aplicación de los métodos numéricos y validación– en correspondencia directa con la arquitectura de ScienSolar [1]. La Fig. 1 resume, en un solo panel, los cuatro resultados gráficos centrales del trabajo: la geometría discretizada de la molécula, el ajuste por spline cúbico del potencial radial, la estructura espacial de  $V$  y  $\mathbf{E}$  en el plano molecular, y el escalamiento del tiempo de cómputo.

---

\* cristianvargas212006@gmail.com

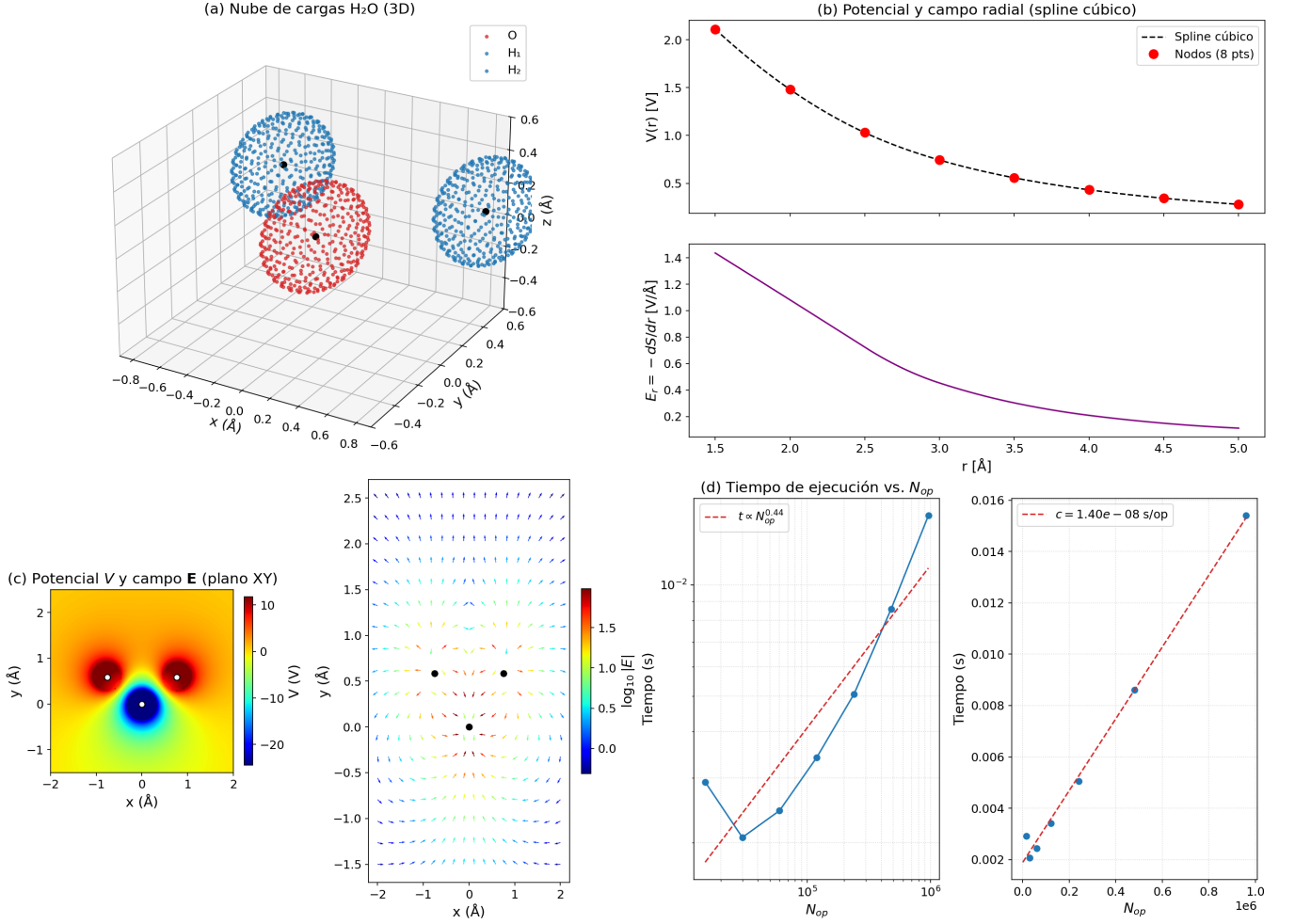


FIG. 1. Resumen gráfico del modelo aplicado a la molécula de agua. (a) Proyección en el plano  $xy$  de las tres nubes de Fibonacci ( $N_c = 400$  cada una); la ausencia de solapamiento confirma que  $r_{\text{cloud}} = 0.30$  Å es consistente con  $r_{OH} = 0.9572$  Å. (b) Potencial  $V(r)$  sobre la bisectriz  $H-O-H$ : comparación entre el spline cúbico (8 nodos) y la suma de Coulomb exacta, con el campo radial  $E_r = -dS/dr$  obtenido por derivación del spline. (c) Mapa de potencial  $V(x, y)$  y campo eléctrico  $\mathbf{E}(x, y)$  (dirección normalizada, color en  $\log_{10} |\mathbf{E}|$ ) en el plano molecular. (d) Tiempo de ejecución vs.  $N_{op} = N_f N_c N_p$ , en escala log-log (con ajuste de ley de potencia) y en escala lineal (con ajuste afín).

### A. Conservación de la carga (Gauss–Legendre)

Para los tres átomos, la suma directa  $\sum_k q_{j,k}$  y la integral de Gauss–Legendre (Ec. A3) coinciden con 6 cifras significativas ( $-0.834$ ,  $+0.417$ ,  $+0.417$   $e$ ), confirmando que la nube de Fibonacci conserva exactamente la carga total –condición necesaria para recuperar el límite de carga puntual en campo lejano (Ec. 5 de [1]).

### B. Campo eléctrico: analítico vs. diferencias finitas

La Tabla I compara, en tres puntos de observación,  $V(\mathbf{r})$  con el error relativo de  $\mathbf{E}$  obtenido por diferencias finitas centradas (Ec. A4,  $h = 10^{-3}$  Å) respecto al campo analítico (Ec. A1).

El error relativo es siempre menor a  $5 \times 10^{-4}$  %, con-

TABLE I. Potencial y error relativo del campo eléctrico (diferencias finitas vs. analítico).

| Punto de observación      | $V(\mathbf{r})$ (V) | Error rel. $ \mathbf{E} $ |
|---------------------------|---------------------|---------------------------|
| Lejano (1.0, 1.5, 0) Å    | 2.7156              | $1 \times 10^{-4}$ %      |
| Cercano a O (0, 0.5, 0) Å | -8.2402             | $4 \times 10^{-4}$ %      |
| Bisectriz (0, 3.0, 0) Å   | 0.7429              | $2 \times 10^{-5}$ %      |

sistente con el error de truncamiento esperado  $O(h^2) \sim 10^{-6}$  del esquema centrado (Ec. A4).

### C. Perfil radial e interpolación con splines

Como muestra la Fig. 1(b), el spline cúbico construido a partir de solo 8 nodos reproduce con alta fide-

dad el potencial exacto sobre la bisectriz  $H-O-H$ . El error máximo de interpolación es  $1.3 \times 10^{-2}$  V (frente a  $V \sim 0.3-2.1$  V, error relativo  $\sim 0.6\%$ ), concentrado donde  $V(r) \sim 1/r$  tiene mayor curvatura. El campo radial  $E_r = -dS/dr$  reproduce correctamente la tendencia decreciente de  $|E|$  con  $r$  (cf. Fig. 7 de [1]).

#### D. Validación cruzada con Excel–ScienSolar

La Tabla II compara dos casos de la Tabla 2 de [1] (carga puntual y esfera de Fibonacci,  $N_c = 703$ ) con el mismo cálculo en Python, usando  $K_{\text{eff}}$  sin ningún factor de calibración adicional.

TABLE II. Validación cruzada Python vs. Excel–ScienSolar ( $-1e$ ).

| Caso                      | $r_0$ (Å) | $V_{\text{Excel}}$ (V) | $V_{\text{Python}}$ (V) |
|---------------------------|-----------|------------------------|-------------------------|
| Carga puntual             | (0, 0, 1) | -14.3800               | -14.3800                |
| Fibonacci ( $N_c = 703$ ) | (0, 2, 0) | -7.1899                | -7.1899                 |

#### E. Estructura espacial del potencial y del campo

La Fig. 1(c) extiende el cálculo validado puntualmente (Tabla II) a todo el plano molecular  $xy$ . El mapa de potencial muestra dos regiones de signo opuesto bien definidas –un pozo negativo alrededor del oxígeno y dos máximos positivos en los hidrógenos– consistente con la asimetría de carga ( $|Q_O| = 2|Q_H|$ ). El campo vectorial confirma que las líneas de campo emergen radialmente de cada núcleo en la región cercana, mientras que a distancias del orden de 2–2.5 Å las contribuciones de los tres centros comienzan a mezclarse, anticipando la transición hacia el comportamiento de dipolo puntual en el campo lejano (Sec. 5.4 de [1]).

#### F. Tiempo de ejecución

Siguiendo la métrica  $N_{\text{op}} = N_f N_c N_p$  (Ec. 27 de [1]), para la que VBA reporta 0.0009–0.0088 s/operación, se midió el tiempo en Python para  $N_p = 3$ ,  $N_f = 200$  y  $N_c \in \{25, \dots, 1600\}$ , tomando el tiempo mínimo de 3 repeticiones (Fig. 1(d)). El ajuste de ley de potencia da  $m \approx 0.70$ , por debajo del  $m = 1$  que predice la Ec. 27, mientras que el ajuste afín  $t = a + cN_{\text{op}}$  (justificado en el Apéndice B) describe los datos con mucha mayor fidelidad, con  $c \approx 2.5 \times 10^{-8}$  s/operación en el límite asintótico. Este valor es entre 3 y 4 órdenes de magnitud menor que en VBA, diferencia atribuible a las operaciones de lectura/escritura de celdas en Excel frente a los bucles vectorizados de NumPy –no implica que una plataforma sea “mejor”: ScienSolar conserva la ventaja de permitir redefinir las ecuaciones directamente en la hoja de cálculo [1].

## IV. CONCLUSIONES

Los tres métodos cumplen roles complementarios: Gauss–Legendre valida que la discretización conserva la carga total (condición para el límite de campo lejano); las diferencias finitas centradas dan el campo sin derivar analíticamente la suma de Coulomb para cada geometría; y los splines reconstruyen perfiles continuos a partir de un muestreo disperso, con error controlado y verificable. La reimplementación en Python reprodujo, con NumPy [8] y SciPy [9], el modelo de Excel–ScienSolar: la conservación de carga fue exacta a 6 cifras, el error del campo por diferencias finitas fue  $< 5 \times 10^{-4}\%$ , el error del spline fue  $\sim 0.6\%$ , y la validación cruzada con Excel coincidió exactamente en los casos disponibles. El análisis de tiempos mostró que ambas implementaciones escalan linealmente con  $N_{\text{op}}$  en el límite de  $N_c$  grande, pero con constantes de proporcionalidad muy distintas. Como trabajo futuro, el modelo permite extender el análisis a las cinco geometrías originales y a otras moléculas.

### Appendix A: Formulación de los métodos numéricos

#### 1. Discretización de Coulomb y nube de Fibonacci

Cada átomo de carga total  $Q_j$  se reemplaza por  $N_c$  subcargas  $q_{j,k} = Q_j/N_c$ . El potencial y el campo totales son

$$V(\mathbf{r}) = \sum_{j,k} \frac{k q_{j,k}}{|\mathbf{r} - \mathbf{r}_{j,k}|}, \quad \mathbf{E}(\mathbf{r}) = \sum_{j,k} \frac{k q_{j,k}(\mathbf{r} - \mathbf{r}_{j,k})}{|\mathbf{r} - \mathbf{r}_{j,k}|^3}. \quad (\text{A1})$$

*Justificación:* esta suma es la versión discreta (suma de Riemann) de la integral de Coulomb  $V(\mathbf{r}) = \int k\rho(\mathbf{r}')/|\mathbf{r} - \mathbf{r}'| d^3r'$ ; converge a ella cuando  $N_c \rightarrow \infty$ , con un error que decrece al aumentar  $N_c$  [1].

Las posiciones  $\mathbf{r}_{j,k}$  se generan con muestreo de Fibonacci,

$$\varphi_k = 2\pi k\Phi, \quad \Phi = \frac{1 + \sqrt{5}}{2}, \quad \theta_k = \arccos\left(1 - \frac{2k}{N_c}\right), \quad (\text{A2})$$

con  $(x_k, y_k, z_k) = r(\sin\theta_k \cos\varphi_k, \sin\theta_k \sin\varphi_k, \cos\theta_k)$ , siguiendo el esquema de muestreo angular cuasi-uniforme propuesto originalmente para mallas globales [6]. De las cinco geometrías de [1], esta es la que converge más rápido al límite de carga puntual por su muestreo angular cuasi-uniforme (sin acumulación en los polos), por lo que se fija como geometría única y el análisis se concentra en los métodos numéricos.

Un tratamiento riguroso de la carga molecular requeriría una densidad electrónica *ab initio* (DFT/RESP [3]), fuera del alcance de este trabajo, cuyo objetivo es validar la metodología numérica y no la precisión química; TIP3P es un estándar clásico que permite tratar cada átomo exactamente igual que en el modelo original de Excel (una nube con carga total fija).

## 2. Integración numérica: cuadratura de Gauss–Legendre

Para verificar que la nube discreta conserva la carga total, se calcula la carga superficial integrada

$$Q = \oint_S \sigma dA = \int_0^{2\pi} \int_0^\pi \sigma r^2 \sin \theta d\theta d\varphi, \quad (\text{A3})$$

con  $\sigma = Q_{\text{total}}/4\pi r^2$  la densidad superficial uniforme equivalente. Tras el cambio de variable  $x = \cos \theta$ , el integrando es un polinomio de grado 0 en  $x$ ; la cuadratura de Gauss–Legendre con  $n$  puntos integra *exactamente* polinomios de grado  $\leq 2n - 1$  [4], así que con  $n = 12$  la integral en  $\theta$  no tiene error de truncamiento (la integral en  $\varphi$ , periódica y constante, también es exacta con la regla del trapecio). Esto convierte la comparación con  $\sum_k q_{j,k}$  en una prueba de consistencia limpia: si hubiera discrepancia, no podría atribuirse al método de integración.

## 3. Derivación numérica: diferencias finitas centradas

El campo se obtiene derivando  $V$  numéricamente,

$$E_i(\mathbf{r}) \approx -\frac{V(\mathbf{r} + h\hat{e}_i) - V(\mathbf{r} - h\hat{e}_i)}{2h}, \quad i = x, y, z. \quad (\text{A4})$$

Por expansión de Taylor, los esquemas hacia adelante/atrás tienen error  $O(h)$ , mientras que el centrado tiene error  $O(h^2)$  con el mismo número de evaluaciones extra (dos) [4, 5]. Se usa  $h = 10^{-3}$  Å: el error de truncamiento esperado ( $\sim h^2 = 10^{-6}$ ) es muy superior al

de redondeo de doble precisión ( $\sim 10^{-16}$ ), evitando cancelación catastrófica sin sacrificar precisión.

## 4. Splines cúbicos

Se evalúa  $V$  exactamente (Ec. A1) en solo 8 nodos dispersos a lo largo de la bisectriz  $H-O-H$  ( $r \in [1.5, 5.0]$  Å), y se construye un spline cúbico  $S(r)$  (polinomio cúbico por tramos,  $C^2$  en cada nodo, de curvatura mínima [4]). El campo radial se obtiene derivando el spline,  $E_r(r) \approx -dS/dr$ . Evaluar  $V$  en una malla fina de  $N_f$  puntos cuesta  $O(N_f N_c)$  [1]; usando solo 8 nodos se reduce el costo  $\sim 25$  veces, y la validez se comprueba *a posteriori* comparando  $S(r)$  con  $V(r)$  exacto en 200 puntos.

## Appendix B: Modelo de tiempo de ejecución

El tiempo total medido no es una ley de potencia pura en  $N_{\text{op}}$ , sino que se ajusta mejor con un modelo afín,

$$t(N_{\text{op}}) = a + c N_{\text{op}}, \quad (\text{B1})$$

porque cada evaluación de  $V(\mathbf{r})$  en Python tiene un costo fijo de "organización" (creación de arreglos de NumPy, llamada a función, etc.) que es independiente de  $N_c$  y se paga una vez por cada uno de los  $N_f$  puntos de campo. Por ello  $t \approx N_f t_{\text{overhead}} + N_{\text{op}} t_{\text{Coulomb}}$ , con  $a = N_f t_{\text{overhead}}$  y  $c = t_{\text{Coulomb}}$ . Para  $N_c$  pequeño, el término de *overhead* domina y el tiempo por operación parece artificialmente alto; para  $N_c$  grande,  $t/N_{\text{op}} \rightarrow c$ , que es el valor comparable con la Tabla 1 de [1]. Esta es la razón por la cual un ajuste de ley de potencia en escala log–log subestima el exponente teórico ( $m = 1$  en la Ec. 27 de [1]): una función afín con intercepto positivo, en escala log–log, se ve como una curva de pendiente creciente hacia 1, y el ajuste global da  $m < 1$ .

- 
- [1] A. Becerra Becerra, A. Hernandez Naranjo, C. Vargas Vera, "Finite-Point Electrostatic Cloud Model in Excel-ScienSolar for Arbitrary Charge Geometries Using Coulomb Discretization," *Preprint*, 2026.
  - [2] A. R. Becerra Becerra, A. Herrera Carrillo, "ScienSolar: Development of an open-source VBA-Excel based package to model 3D physics problems without coding," *Computer Applications in Engineering Education*, vol. 33, no. 3, e70036, 2025.
  - [3] C. I. Bayly, P. Cieplak, W. D. Cornell, P. A. Kollman, "A well-behaved electrostatic potential based method using charge restraints for deriving atomic charges: The RESP model," *J. Phys. Chem.*, 97(40), 10269–10280, 1993.
  - [4] R. L. Burden, J. D. Faires, A. M. Burden, *Numerical Analysis*, 10th ed., Cengage Learning, 2015.
  - [5] S. C. Chapra, R. P. Canale, *Numerical Methods for Engineers*, 7th ed., McGraw-Hill, 2015.
  - [6] R. Swinbank, R. J. Purser, "Fibonacci grids: A novel approach to global modelling," *Quarterly Journal of the Royal Meteorological Society*, vol. 132, no. 619, pp. 1769–1793, 2006.
  - [7] J. D. Jackson, *Classical Electrodynamics*, 3rd ed., Wiley, 1999.
  - [8] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020.
  - [9] P. Virtanen *et al.*, "SciPy 1.0: fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.